Handling 2+ Degrees of Freedom (DOF)

If the final exam provides a system with two or more coordinates (e.g., a cart-pendulum with x and θ, or a double pendulum with θ_1 and θ_2), the Euler-Lagrange method expands naturally.

1. Lagrangian for 2-DOF

The Lagrangian remains $L = K - P$, but now both K and P are functions of multiple variables:

$$L(q_1, q_2, \dot{q}_1, \dot{q}_2)$$

2. System of Equations

You will have one Euler-Lagrange equation for **each** coordinate: 1. $\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{q}_1}\right) - \frac{\partial L}{\partial q_1} = Q_1$ 2. $\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{q}_2}\right) - \frac{\partial L}{\partial q_2} = Q_2$

3. Adapting the Symbolic Script (`symbolic_dynamics_deriver.py`)

To solve a 2-DOF system using the provided Python tool, make these changes:

Update `__init__`

```
# Define two coordinates
self.q1 = sp.Function('q1')(self.t)
self.q2 = sp.Function('q2')(self.t)

# Define derivatives
self.q1_dot = sp.diff(self.q1, self.t)
self.q2_dot = sp.diff(self.q2, self.t)
self.q1_ddot = sp.diff(self.q1_dot, self.t)
self.q2_ddot = sp.diff(self.q2_dot, self.t)
```

Update `derive_eom`

Modify the solver to solve for both accelerations simultaneously:

```
# Form both equations
eq1 = sp.Eq(ddt_dL_dq1_dot - dL_dq1, self.Q1)
eq2 = sp.Eq(ddt_dL_dq2_dot - dL_dq2, self.Q2)

# Solve for both second derivatives
solutions = sp.solve([eq1, eq2], [self.q1_ddot, self.q2_ddot])
```

4. State-Space Implications

A 2-DOF system will result in 4 states: $x = [q_1, \dot{q}_1, q_2, \dot{q}_2]^T$. When implementing `f(x, u)` in `dynamics.py`: - `xdot[0] = x[1]` (Velocity 1) - `xdot[1] = solutions[q1_ddot]` (Acceleration 1) - `xdot[2] = x[3]` (Velocity 2) - `xdot[3] = solutions[q2_ddot]` (Acceleration 2)